Rapport Travail 8 - Group Dih

REY Dorian DURAND Eliot BASILE Francesco-Pio BOUREUX Nathan

Faculté des Sciences
Université de Montpellier

Avril 2026

Table des matières

1 Entraînement du modèle DistilBERT . . . 3
1.1 Source de nos données et prétraitement . . . 3
1.2 Premier fine tuning . . . 3
1.3 Améliorations : Prétraitement supplémentaire . . . 4
1.4 Test réel du modèle . . . 5
1.5 Optimisation supplémentaire . . . 6
1.6 Code Google colab et modèle . . . 6
1.7 Objectif suivant . . . 6
2 Agrégateur de Vulnérabilités . . . 7
2.1 Hiérarchie des 5 Sources . . . 7
2.2 Normalisation des CPE & Mapping . . . 7
2.3 Nouvelle Architecture . . . 7
2.4 Objectif suivant . . . 8
3 Annexe . . . 9

1 Entraînement du modèle DistilBERT

1.1 Source de nos données et prétraitement

Suite à la mise en place initiale de la pipeline de base avec un modèle, nous devons maintenant entraîner notre modèle. Nous nous sommes appuyés sur les travaux de M. Belhout comme base mais avons amélioré certains points.

La provenance des données, la source est la même que M. Belhout, soit les fichiers JSON de la NVD (Fig. 2). Cette fois-ci, nous prélèverons les CVE entre 2015 et 2026, les CVE avant 2015 n'ayant quasiment pas de notes en CVSS 3.0 ou 3.1. Nous gardons uniquement les CVE de type « Primary », soit les CVE dont le score CVSS a été noté par la NVD, et non d'autres sources comme GitHub.

Le prétraitement se fait à l'aide d'un script python, nous récupérons la description ainsi que les 8 métriques, directement converties en chiffre (0, 1, 2, 3). Si une des métriques n'est pas remplie, ou si le score 3.0 ou 3.1 n'est pas disponible, alors la CVE n'est pas retenue. La qualité de la donnée d'entraînement est ainsi maximale. Le fichier CSV final fait un peu moins de 190 000 lignes.

Le papier de recherche affichant le niveau de performance le plus élevé (Fig. 3) pour notre tâche avait un dataset de 45 000 lignes. Dans notre cas, nous utiliserons 50 000 lignes, en piochant de manière aléatoire parmi les 190 000 lignes de notre dataset, permettant une répartition égale des données. Ce chiffre permet de s'aligner théoriquement à leur performance très élevée (~88% d'accuracy global), en plus de ne pas avoir un entraînement trop long (environ 18 minutes par epochs).

1.2 Premier fine tuning

Après avoir récupéré un modèle DistilBERT pré-entraîné et possédant une compréhension grammaticale et syntaxique profonde de l'anglais. Nous allons ainsi le fine tuner pour l'entraîner à prédire nos 8 métriques. 80% des données seront utilisées pour l'entraînement et 20% pour la validation. Nous avons ainsi fait tourner notre modèle sur Google Colab dont voici les résultats sur le tout premier entraînement d'initialement de 10 epochs réduit à 8 :

Métrique	E1	E2	E3	E4	E5	E6	E7	E8
Accuracy Train	0.8469	0.9013	0.9150	0.9245	0.9339	0.9422	0.9501	0.9573
Accuracy Val.	0.8924	0.9064	0.9094	0.9094	0.9105	0.9104	0.9092	0.9098
F1 Train	0.8346	0.8967	0.9118	0.9219	0.9318	0.9407	0.9489	0.9565
F1 Val.	0.8859	0.9030	0.9065	0.9072	0.9085	0.9085	0.9076	0.9084
Loss Train	3.1710	2.2191	1.9297	1.7198	1.5232	1.3283	1.1574	1.0036
Loss Val.	2.3770	2.1089	2.0409	2.0415	2.0660	2.1574	2.2747	2.3521

Tableau 1. – Résultats du modèle baseline

Notre modèle est le meilleur à la 3ème époque, symbolisé par la Validation Loss la plus basse (plus elle est basse, mieux c'est), qui est la métrique la plus importante dans notre cas. C'est celui-ci que nous allons sauvegarder et intégrer à notre pipeline. La Fig. 4 (onglet « Baseline | No pretreating ») présente tous les résultats détaillés des modèles. Pour ce modèle, certains hyperparamètres (learning rate, batch size) ont été choisis arbitrairement à l'aide de conseils d'internet et de l'IA, il est ce que l'on appelle un « modèle de base » (ou « baseline »), et nous ferons référence à ce dernier via cette appellation dans le

reste du rapport de travail.

Ces premiers résultats sont encourageants, la F1 est très proche de l'accuracy, symbolisant un non sur-apprentissage. Cependant, la validation loss et la validation accuracy stagnent tandis que la train continue de descendre à partir de la 4ème epoch, signe d'un sur apprentissage, notre modèle semble rencontrer un plafond à 90% de validation accuracy.

En conclusion de ce premier fine-tuning, notre modèle est performant en plus d'être hébergé localement (avec l'avantage de la sécurité et l'intégrité des données, l'indépendance et la rapidité). Maintenant que la pipeline est mise en place, il nous suffit simplement de continuer à entraîner notre modèle et le changer dès que nous aurons une meilleure version.

1.3 Améliorations : Prétraitement supplémentaire

Nous allons tenter, même s'il est déjà très bon, d'améliorer encore le modèle, car certains points peuvent être améliorés, notamment le prétraitement et l'utilisation d'Optuna.

Concernant le prétraitement, nous allons appliquer les méthodes standards de prétraitement de texte :
- Mettre le texte en minuscule
- Enlever les multis-espaces, les multis-virgules ou points virgules
- Formater le HTML
- Enlever les caractères spéciaux

Nos descriptions de CVE contiennent souvent les chemins de fichiers, des adresses mémoires ou bien des numéros de versions, nous allons ainsi :
- Formater tous les chemins, versions, id de CVE ou adresse mémoire en : cve_id_token, mem_addr_token, file_path_token, version_token
- Formatage des dates en : date_token
- Formatage des url et des email en : url_token, email_token

Afin de véritablement voir si cela a un impact, voilà les résultats obtenus après entraînement avec les données prétraitées en entrée :

Métrique	E1	E2	E3	E4	E5	Meilleur baseline sans pré-traitement
Accuracy Train	0.8479	0.9013	0.9145	0.9244	0.9336	0.9150
Accuracy Val.	0.8940	0.9064	0.9087	0.9108	0.9100	0.9094
F1 Train	0.8357	0.8967	0.9111	0.9218	0.9316	0.9118
F1 Val.	0.8875	0.9032	0.9053	0.9084	0.9078	0.9065
Loss Train	3.1774	2.2216	1.9343	1.7231	1.5188	1.9297
Loss Val.	2.3718	2.1247	2.0727	2.0620	2.1033	2.0409

Tableau 2. – Résultats du modèle baseline avec pré traitement

Seulement 5 epochs ici, car la val loss remonte dès l'epoch 5. Le meilleur modèle est ici celui de l'epoch 4, qui a la loss la plus basse. Si on le compare à notre meilleur modèle avant le prétraitement, il a quasiment la même loss (+0,02) mais possède une meilleure accuracy val (+0,14) et F1 val. (+0,19), signifiant qu'il est très légèrement meilleur globalement.

Ceci est une très bonne nouvelle, notre modèle n'a pas perdu en qualité alors qu'il est maintenant forcé à apprendre le vrai sens sémantique des descriptions. Il n'est plus influencé par des artefacts (versions, chemins, identifiants), mais bien par la logique profonde des descriptions, garantissant une meilleure performances pour les nouvelles CVE. Les résultats détaillés sont dans la Fig. 4 (onglet « Baseline | Pretreating »)

1.4 Test réel du modèle

A ce stade, nous nous posions la question de savoir si le modèle était vraiment bon sur des exemples qu'il n'a jamais vus, mais dans des conditions réelles. Nous avons ainsi lancé un autre entraînement, cette fois-ci avec uniquement les CVE entre 2015 et 2023 (réduction du dataset), puis le tester au hasard sur des descriptions de CVE entre 2024 et 2026. Le détail brut des résultats d'entraînements, du modèle choisi et du test des CVE sont disponibles à la Fig. 4 (respectivement onglet « Baseline | Pretreating | With only 2015 - 2023 CVE » et « Test of model with only 2015 - 2023 CVE »).

CVE ID	Vecteur NVD (Réel)	Score NVD	Vecteur Modèle	Score Modèle
CVE-2024-22288	AV:N/AC:L/PR:N/ UI:R/S:C/C:L/I:L/A:L	7.1	AV:N/AC:L/PR:N/ UI:R/S:C/C:L/I:L/A:N	6.1
CVE-2024-27073	AV:L/AC:L/PR:L/ UI:N/S:U/C:N/I:N/A:H	5.5	AV:L/AC:L/PR:L/ UI:N/S:U/C:N/I:N/A:H	5.5
CVE-2024-46591	AV:N/AC:L/PR:N/ UI:N/S:U/C:N/I:N/A:H	7.5	AV:N/AC:L/PR:N/ UI:N/S:U/C:N/I:N/A:H	7.5
CVE-2025-5799	AV:N/AC:L/PR:L/ UI:N/S:U/C:H/I:H/ A:H	8.8	AV:N/AC:L/PR:N/ UI:N/S:U/C:H/I:H/ A:H	9.8
CVE-2025-4394	AV:P/AC:L/PR:N/ UI:N/S:U/C:H/I:H/ A:H	6.8	AV:P/AC:L/PR:N/ UI:N/S:U/C:H/I:H/A:N	6.1
CVE-2025-10442	AV:N/AC:L/PR:N/ UI:N/S:U/C:H/I:H/ A:H	9.8	AV:N/AC:L/PR:N/ UI:N/S:U/C:H/I:H/ A:H	9.8
CVE-2026-23072	AV:L/AC:L/PR:L/ UI:N/S:U/C:N/I:N/A:H	5.5	AV:L/AC:L/PR:L/ UI:N/S:U/C:N/I:N/A:H	5.5
CVE-2026-0007	AV:L/AC:L/PR:N/ UI:R/S:C/C:H/I:H/A:H	8.6	AV:L/AC:L/PR:L/ UI:N/S:U/C:H/I:H/ A:H	7.8
CVE-2026-1663	AV:N/AC:L/PR:L/ UI:N/S:U/C:N/I:L/A:N	4.3	AV:N/AC:L/PR:L/ UI:N/S:U/C:N/I:L/A:N	4.3

Tableau 3. – Test du modèle sur des vulnérabilités récentes qui lui sont inconnues

Ces résultats confirment la robustesse de notre modèle : l'écart sur le score final n'excède jamais 1.0 (sachant qu'une seule mauvaise prédiction de métrique peut drastiquement changer la note). Il parvient notamment à identifier correctement le vecteur d'attaque « Physical » (P), pourtant classe la plus minoritaire du jeu de données. Enfin, malgré la densité technique de la CVE-2026-23072 (surabondance d'adresses mémoire et de chemins de fichiers), le modèle prédit sans erreur l'intégralité des huit métriques. Cette capacité de généralisation valide notre approche de prétraitement.

1.5 Optimisation supplémentaire

Une optimisation supplémentaire est la minimisation des données envoyées aux clients. Ainsi la route permettant de récupérer une CVSS a été supprimée au profit d'une fonction interne. Ci-dessous un schéma illustrant l'implémentation :

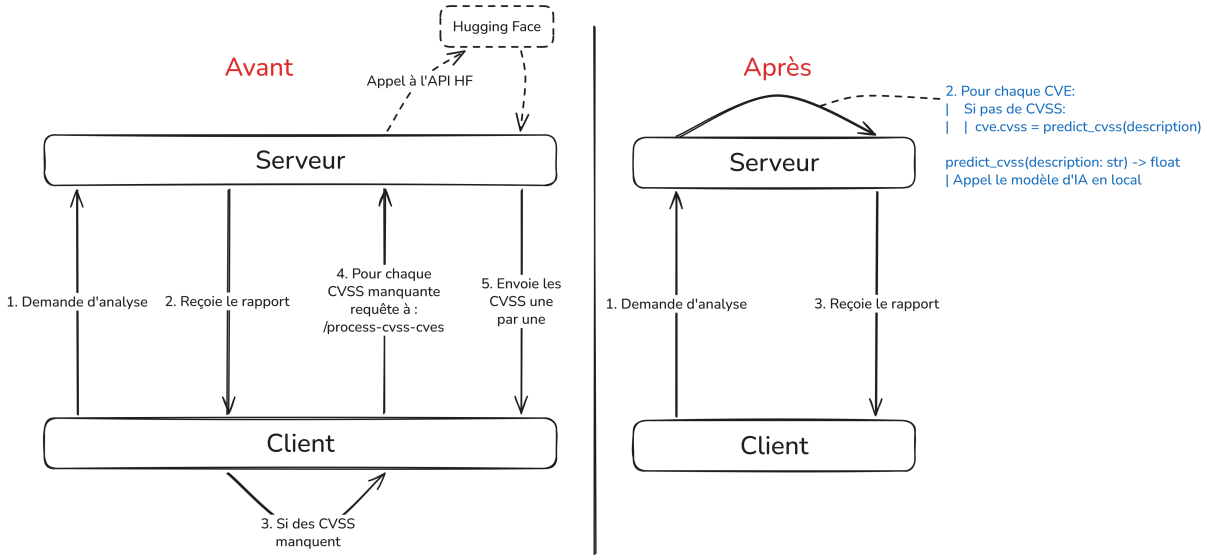

Fig. 1. – Schéma illustrant les requêtes HTTP entre client serveur avant et après optimisation

1.6 Code Google colab et modèle

Le code Google Colab (Notebook Python) est disponible à la Fig. 5.

Le lien de téléchargement de notre meilleur modèle (Baseline avec prétraitement) est disponible à la Fig. 6.

1.7 Objectif suivant

La seule manière d'encore améliorer notre modèle, serait d'optimiser ses hyper paramètres, en utilisant Optuna ainsi que d'entraîner le modèle sur l'intégralité de notre dataset, afin qu'il voit un maximum d'exemples possible. Ceci sera fait si jugé nécessaire lors de la prochaine réunion car cela va prendre un temps non négligeable. Nous pourrions également rééquilibré les classes dans notre jeu de données.

2 Agrégateur de Vulnérabilités

Le système utilise une stratégie de failover hiérarchisée. Le moteur interroge les sources dans un ordre de priorité strict.

2.1 Hiérarchie des 5 Sources

1. EUVD (European Vulnerability Database) : Source primaire pour les données européennes, privilégiée pour sa précision sur les versions affectées.
2. OSV (Open Source Vulnerabilities) : Premier fallback, spécialisé dans les écosystèmes open-source (Maven, NPM, PyPI).
3. NVD (National Vulnerability Database) : Base historique globale, utilisée principalement comme index de mapping CPE → CVE.
4. GitHub Security Advisories : Source agile pour les vulnérabilités récentes.
5. JVN (Japan Vulnerability Notes) : Source régionale pour couvrir les vulnérabilités signalées par le CERT japonais.

Bien que les deux premières sources suffisent à couvrir plus de 95 % des cas d'usage, l'intégration des autres a été mise en place pour garantir un niveau d'exhaustivité et de sécurité maximal.

Nous suivons ce flux de donnée :

- Si « VULNÉRABLE » trouvé : Écriture en base de données et arrêt.
- Si « NON AFFECTÉ » (définitif) : Arrêt immédiat (Composant sain).
- Si « NON TROUVÉ » : Passage à la source suivante.
- Si « ÉPUISÉ » : Marquage comme « UNKNOWN ».

2.2 Normalisation des CPE & Mapping

L'un des principaux défis était le décalage entre les noms de paquets Maven et les nomenclatures NVD :

- Exemple Jackson : Transformation de `com.fasterxml.jackson.core` vers le vendor `fasterxml` pour éviter les faux négatifs.
- Exemple BouncyCastle : Gestion des variantes `bcprov-jdk15on` et `bcprov-jdk18on`.

Pour résoudre cette problématique (temporairement), nous avons développé notre propre dictionnaire de conversion. Celle-ci convertit et standardise automatiquement les nomenclatures. Mais la prise en compte d'une multitude d'écosystèmes et de cas particuliers implique une gestion continue pour anticiper et limiter l'émergence de faux positifs ou négatifs.

Cependant, cette solution n'est ni extensible ni pratique, et nous en connaissons une autre, dont l'implémentation est l'un des objectifs principaux de la prochaine phase de travail.

2.3 Nouvelle Architecture

Pour supporter la mise en place de ce pipeline, l'architecture globale de l'application a subi une refonte majeure, en particulier au niveau du backend.

Par conséquent, le système expose désormais sa propre API RESTful, propulsée par FastAPI. Cette architecture modulaire permet d'interroger directement le moteur de scan et met à disposition une documentation interactive (Swagger UI) consultable localement via : `http://localhost:8000/docs`.

2.4 Objectif suivant

Afin de poursuivre le développement et d'augmenter la résilience de l'agrégateur, nous avons entamé l'implémentation du modèle d'Intelligence Artificielle détaillé précédemment.

Ce modèle a été conçu pour agir comme un sixième et ultime niveau de fallback. Il n'interviendra qu'en dernier recours, dans l'hypothèse où un composant resterait classé « Inconnu » après avoir interrogé les cinq sources du pipeline (EUVD, OSV, NVD, GitHub, JVN).

À l'heure actuelle, le couplage de ce modèle à l'architecture principale est implémenté. Mais cette fonctionnalité demeure en phase expérimentale et n'a pas encore été testée en conditions réelles. Elle devra faire l'objet d'une campagne de tests rigoureuse avant tout déploiement, afin d'évaluer son taux de fiabilité (faux positifs/négatifs) et son impact sur les performances globales du système.

3 Annexe

2. https://nvd.nist.gov/vuln/data-feeds#divJson20Feeds
3. https://arxiv.org/abs/2111.08510
4. https://docs.google.com/document/d/11eCs1nqLkFaR1OKISUttD2aPfVNNbwsUF24GuN8RB_w/edit?usp=sharing
5. https://urlix.me/cvss-colab-code
6. https://urlix.me/cvss-models